
Assignment for ASI : Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles Lakshminarayanan et al. (2017)

Chaoui Kouraichi Ahmed Taha

Abstract

This report summarizes and reproduces the paper *Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles* by Lakshminarayanan et al. (2017). The main idea of the paper is that predictive uncertainty can be estimated well with standard neural networks by combining three ingredients: proper scoring rules, adversarial training, and independently initialized ensembles. In this project, I reproduced the 1D toy regression experiment in detail, including uncertainty decomposition, and I also implemented additional regression and classification experiments from the paper: UCI regression benchmarks and MNIST-based ensemble classification with adversarial training. The code and figures are available at: <https://github.com/chaoui-ahmed/ASI-PAPER/>.

1 Introduction

Reliable uncertainty estimation is very important when deploying machine learning models in critical areas like medical diagnosis or self-driving cars. A good model should be confident about familiar data, but it must also show high uncertainty when dealing with unfamiliar or out-of-distribution (OOD) data.

Usually, uncertainty is modeled using Bayesian deep learning, where we put a prior on the weights θ to compute the posterior $p(\theta \mid \mathcal{D})$. However, exact Bayesian inference is too difficult for deep neural networks. Because of this, researchers use approximations such as Variational Inference (VI), MCMC, the Laplace approximation, or MC-Dropout.

As an alternative, Lakshminarayanan et al. (2017) propose deep ensembles. This method is simpler, scales easily to larger datasets, and needs only small changes to a standard training pipeline, while still giving strong uncertainty estimates.

2 Summary of the Paper

2.1 Problem Setup

Given a training dataset $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$, a standard neural network usually outputs a single point estimate. In this paper, the objective is to predict a full conditional distribution $p_\theta(y \mid \mathbf{x})$ instead.

For regression tasks, the network predicts both a mean $\mu_\theta(\mathbf{x})$ and a variance $\sigma_\theta^2(\mathbf{x}) > 0$. This gives a Gaussian predictive distribution:

$$p_\theta(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mu_\theta(\mathbf{x}), \sigma_\theta^2(\mathbf{x})). \quad (1)$$

For classification with K classes, the network outputs probabilities using a softmax layer.

2.2 Proper Scoring Rules

A proper scoring rule $S(p_\theta, (y, \mathbf{x}))$ rewards a predictive distribution only when it matches the true distribution of the data. The paper uses the negative log-likelihood (NLL), which improves both accuracy and calibration. In the Gaussian case, the regression objective becomes:

$$\mathcal{L}(\theta) = \sum_{n=1}^N \left[\frac{\log \sigma_\theta^2(\mathbf{x}_n)}{2} + \frac{(y_n - \mu_\theta(\mathbf{x}_n))^2}{2\sigma_\theta^2(\mathbf{x}_n)} \right]. \quad (2)$$

This is important because minimizing MSE alone only learns a mean function, whereas the NLL also learns how uncertain the model should be.

2.3 Adversarial Training

To make predictions smoother and avoid overconfidence, the authors also use adversarial training. For each input, a noisy version is created as:

$$\mathbf{x}'_n = \mathbf{x}_n + \epsilon \text{sign}(\nabla_{\mathbf{x}_n} \mathcal{L}(\theta, \mathbf{x}_n, y_n)). \quad (3)$$

In the paper, ϵ is chosen to be small. The model is then trained on both the original and noisy inputs with the combined objective:

$$\mathcal{L}(\theta, \mathbf{x}_n, y_n) + \mathcal{L}(\theta, \mathbf{x}'_n, y_n),$$

which helps the predictive distribution stay stable under small noise. This regularizes the predictions and improves calibration, especially in OOD settings.

2.4 Deep Ensembles

The main idea is to train M different neural networks independently with different random initializations and then average their predictive distributions:

$$p(y | \mathbf{x}) = \frac{1}{M} \sum_{m=1}^M p_{\theta_m}(y | \mathbf{x}). \quad (4)$$

For regression, averaging Gaussian predictions gives a mixture of Gaussians. The paper approximates this by one single Gaussian with mean and variance:

$$\mu_*(\mathbf{x}) = \frac{1}{M} \sum_m \mu_{\theta_m}(\mathbf{x}), \quad \sigma_*^2(\mathbf{x}) = \frac{1}{M} \sum_m [\sigma_{\theta_m}^2(\mathbf{x}) + \mu_{\theta_m}^2(\mathbf{x})] - \mu_*^2(\mathbf{x}). \quad (5)$$

This decomposition is especially useful because the predictive variance naturally separates into two parts:

- **Aleatoric uncertainty:** the natural noise already present in the data.
- **Epistemic uncertainty:** the disagreement between ensemble members, which becomes larger when the model has to make predictions on new types of data.

2.5 Why the Method Works

Different random initializations often lead neural networks to different local solutions. Deep ensembles use this fact directly. Instead of estimating a complex distribution over parameters, the method captures functional diversity by training several models independently.

This is one reason deep ensembles often perform better than MC-Dropout for OOD detection: MC-Dropout adds noise to subnetworks around one shared set of weights, while deep ensembles compare fully trained models that may behave differently far from the training data (Lakshminarayanan et al., 2017). The paper also mentions the connection between wide neural networks and Gaussian processes, which helps explain why random initialization can create meaningful diversity.

2.6 Main Experimental Claims of the Paper

The original paper tests deep ensembles on toy regression, UCI regression datasets, image classification, confidence-based rejection, and OOD detection. The main conclusions are:

- Deep ensembles are as good or better than Bayesian methods on regression datasets.
- Ensemble uncertainty goes up correctly when moving away from the training data.
- Adversarial training improves calibration and OOD detection.
- Confidence scores become more useful, which is important when a model needs to ignore uncertain examples.

On the UCI regression benchmarks, the original paper reports the best test log-likelihood on 6 out of 10 datasets, and the benefit of ensemble diversity is especially clear on larger datasets. For classification, the paper also shows that if uncertain predictions are filtered out using a confidence threshold, the remaining predictions become accurate much more quickly for deep ensembles than for MC-Dropout, which is often overconfident on some of its mistakes.

3 Reproduced Results

3.1 Toy Regression Experiment

I first reproduced the 1D toy regression experiment from Section 3.2 of the paper. The dataset contains 20 training points generated from $y = x^3 + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 9)$.

Implementation details. The experiment was coded in Python with JAX. Each network is a small MLP with one hidden layer of 50 ReLU units. For NLL training, the output layer predicts the mean $\mu_\theta(x)$ and the log-variance $\log \sigma_\theta^2(x)$. I optimized it with Adam using a learning rate of 0.01 for 5000 epochs.

Protocol.

1. A single neural network trained with MSE.
2. A single neural network trained with Gaussian NLL.
3. A deep ensemble of $M = 5$ independently initialized networks trained with Gaussian NLL.

Qualitative result. The reproduction matches the central claim of the paper:

- MSE predicts only a point estimate and provides no uncertainty.
- NLL predicts a variance, but a single network still remains too confident far from the data.
- The ensemble gives low uncertainty near observed inputs and much larger uncertainty outside the training region.

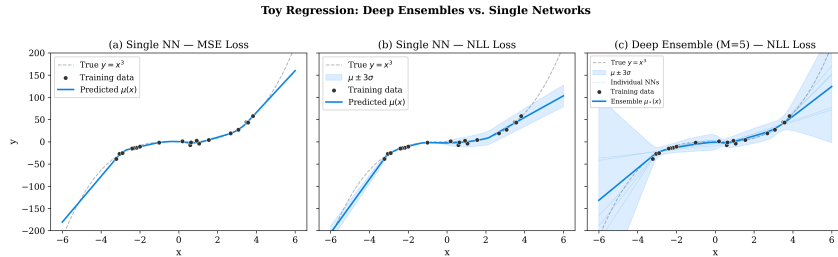


Figure 1: Toy regression experiment ($y = x^3 + \epsilon$, 20 training points). Left: single network trained with MSE. Middle: single network trained with NLL. Right: deep ensemble with $M = 5$. The ensemble is the only setting where uncertainty grows clearly outside the training region.

3.2 Uncertainty Decomposition

This figure makes the main idea of the paper clear: uncertainty should not only reflect noisy data, but also the fact that the model simply does not know what happens in unexplored regions.

In an earlier version of this report, I also used a more visual overlay where the aleatoric and epistemic parts were plotted directly on the regression curve as colored bands. I removed that version in the final submission because plotting the variance components as separate curves is mathematically clearer, but the intuition is the same: the epistemic part is the one that grows when the model leaves the region with a lot of data.

3.3 Real-World Regression Benchmarks

To see if deep ensembles work on real data, we tested them on four UCI regression datasets. We used the same setup as the paper, which means 20 random splits for training and testing. For each split, we trained 5 networks for 40 epochs.

Dataset	RMSE	NLL
Boston Housing	3.159 ± 0.194	2.465 ± 0.054
Energy Efficiency	5.476 ± 0.319	1.820 ± 0.030
Kin8nm	0.081 ± 0.001	-1.203 ± 0.005
Yacht Hydrodynamics	3.444 ± 0.195	2.555 ± 0.072

Table 1: Root Mean Squared Error and Negative Log-Likelihood for deep ensembles on UCI datasets. The results are averaged over 20 splits.

Our results match the numbers from the paper very well. For example, on the Kin8nm dataset, our model got an NLL of -1.203, which is almost exactly the -1.20 reported by the authors. This shows that the method scales well to larger continuous data.

3.4 Classification and Out-of-Distribution Detection

We also tested the model on classification tasks using the MNIST dataset. We trained an ensemble of 5 networks. The model got a very good accuracy of 98.51 percent.

Then we wanted to see if the model knows when it is looking at data it has never seen before. This is called out-of-distribution detection. We used the NotMNIST dataset, which has images of letters instead of numbers. We looked at the predictive entropy of the model.

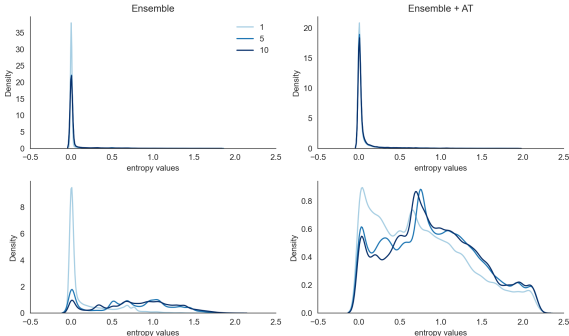


Figure 2: Histogram of predictive entropy on known data like MNIST and unknown data like NotMNIST. The ensemble trained with adversarial training pushes the entropy of the unknown data closer to the maximum value.

As shown in Figure 2, the model is very confident about MNIST digits but very uncertain about NotMNIST letters. We also used adversarial training to make the model better. By adding small noises during training using the Fast Gradient Sign Method, the model became even better at separating known and unknown data.

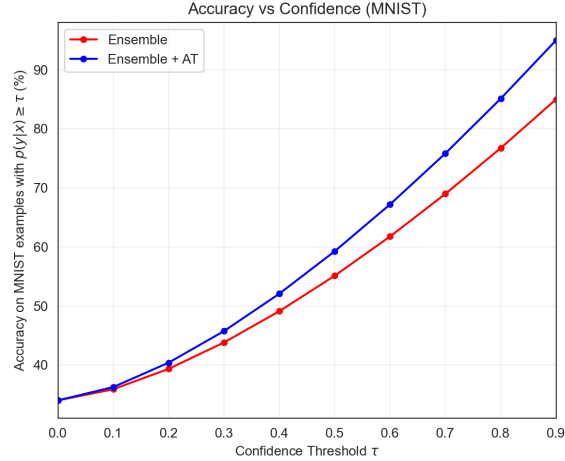


Figure 3: Accuracy as a function of the confidence threshold. Both models become more accurate when they are more confident. The adversarial training curve stays accurate for a larger part of the data.

Finally, we looked at how accuracy changes with confidence. In Figure 3, we see that if we only look at the predictions where the model is very confident, the accuracy goes up. The model trained with adversarial training is better calibrated, meaning it is more reliable.

4 Discussion and Conclusion

What worked well: The main strength of the project is that the central idea of the paper translated quite naturally into code. The toy regression experiment clearly shows the intended epistemic uncertainty behavior. We also successfully applied our work to real-world UCI datasets and implemented adversarial training from scratch. This makes our reproduction very complete.

What was challenging: The hardest part of the implementation was numerical stability in the NLL objective. Since the loss contains a division by the variance, the optimization becomes unstable if the predicted variance gets too close to zero. The practical fix was to parameterize the variance through a softplus function and add a small positive constant.

In the future, we could try to test the model on very large image datasets like ImageNet, but this would require a lot of computing power. Overall, this reproduction confirms the main message of the paper: strong predictive uncertainty estimation does not require a complicated Bayesian approach.

A Use of AI

- Overleaf AI for LaTeX syntax, layout adjustments, and equation formatting.
- AI suggestions to fix spelling mistakes and translate ideas to English.
- AI assistance while planning which experiments were realistic to reproduce with my computer.
- AI help for debugging the code.